

Науково-практичний звіт на тему

Метод розв'язання задачі побудови евклідового мінімального кістякового дерева

Виконала: Весна-Ізбаш Дарія, студентка 3 курсу, групи ТК-31

Анотація. У роботі використано узагальнений метод побудови Евклідового мінімального кістякового дерева (EMST) для заданої множини точок на площині та застосування цього методу до задач оптимізації комунікаційних мереж. В даному випадку застосування виявилось можливим завдяки зведенню геометричної задачі побудови дерева до оптимізаційного пошуку найкоротших зв'язків у графі за допомогою алгоритму Прима.

Вступ

Постановка проблеми. В роботі розглядається один із підходів розв'язання задачі побудови мінімального кістякового дерева в евклідовому просторі в узагальненій постановці. Розв'язки задачі побудови Евклідового мінімального кістякового дерева (EMST) мають велике теоретичне і практичне значення, особливо, що стосується застосування як в інформатиці, так і в прикладних науках.

Наприклад, одним із таких застосувань є задачі розпізнавання образів з використанням методів автоматичної кластеризації даних (алгоритми Single-linkage clustering) та класифікації об'єктів. Для ефективного аналізу багатовимірних метричних просторів необхідно, щоб інформація на вході класифікаторів мала таку властивість, як зв'язність без утворення циклів та мінімальність просторових відстаней. Важливим застосуванням також є побудова ефективних алгоритмів для задач геометричного та графічного моделювання складних явищ і процесів. Зокрема, це стосується моделювання логістичних маршрутів, систем трасування інтегральних мікросхем та проєктування кабельних чи трубопровідних мереж.

Для розв'язання таких задач розробляються узагальнені ефективні алгоритми. Одним із підходів розробки таких алгоритмів є підхід, в основі якого лежить використання геометричних графів близькості, таких як Триангуляція Делоне або Граф Габріеля. Та проблемою застосування класичних підходів є висока обчислювальна складність на великих множинах точок. Тому пошук ефективних підходів, які б дозволяли просто і ефективно розв'язувати як класичну задачу кістякового дерева, так і узагальнену, є актуальним.

Аналіз останніх досліджень. На сьогоднішній день для розв'язання задачі використовують два основних підходи, які ґрунтуються на структурах даних типу збалансованих дерев та матрицях суміжності.

Перший підхід, відомий як підхід «в лоб» (Brute-force), розглядає повний граф, де кожна точка з'єднана з кожною іншою. Перевага даного підходу в тому, що структури даних, які він використовує, є максимально простими і не потребують складних

геометричних перетворень. Недолік даного підходу очевидний: через обчислення відстаней між усіма парами точок зростає час розрахунку до $O(N^2)$, що є неприпустимим для великих масивів даних.

В основі другого підходу лежить використання кінетичних або просторових структур даних, зокрема триангуляції Делоне як суперграфа-кандидата. Він підтримує структуру даних у реальному часі: коли точки фіксовані, будується розріджена сітка трикутників. Такий підхід дає ефективний час пошуку, проте потребує попередньої перебудови геометричної структури.

Зазначимо, що якщо множина S складається з точок в R^2 , то, використовуючи класичні алгоритми Краскала або Прима на повному графі, можна отримати результати за час $O(N^2)$. При зміні розмірності простору або використанні геометричних оптимізацій (наприклад, через діаграми Вороного), цей час можна зменшити до логарифмічного — $O(N \log N)$.

З практичної точки зору, використання складних просторових дерев досить складне у реалізації. Можна використовувати матриці відстаней, не дивлячись на те, що їх використання залишає загальний час погіршеним, але алгоритмічна стійкість та простота коду стають значно вищими. Зазначимо, що в загальній постановці (без суттєвих обмежень на функції метрики) задача є глибоко дослідженою, проте оптимізація її програмного коду сучасними засобами високого рівня залишається важливою задачею.

Новизна та ідея. В даній роботі запропоновано обчислювальний підхід, який узагальнює як саму задачу побудови дерева, так і алгоритм її візуалізації засобами мови Python.

Мета статті. Розробити та програмно реалізувати метод побудови мінімального кістякового дерева для заданої множини точок в евклідовому просторі з можливістю динамічної зміни кількості точок.

Основна частина

Сформулюємо геометричну постановку задачі побудови мінімального дерева.

Постановка задачі побудови дерева. Нехай задана множина S із N точок на площині (в E^2 просторі). Необхідно визначити дерево — граф $T=(V, E)$, де $V = S$, який є зв'язним, не містить циклів, а сума довжин ребер якого є найменшою:

$$\sum_{e \in E} w(e) \rightarrow \min$$

Побудова розв'язку задачі

Нехай $Z_i(x_i, y_i)$ та $Z_j(x_j, y_j)$ - дві довільні точки з множини S . Вага ребра $w(e_{ij})$ між ними визначається як Евклідова відстань за формулою:

$$w(e_{ij}) = \sqrt{(x_j - x_i)^2 + (y_j - y_i)^2}$$

Для обчислення кістякового дерева використовується алгоритм Прима. Алгоритм працює за наступним ітераційним принципом:

1. Вибирається довільна початкова вершина (точка), яка додається до підмножини відвіданих вершин U .
2. На кожному кроці розглядаються всі ребра, що з'єднують вершини з множини U (відвідані) з вершинами, які ще не входять до U (невідвідані).
3. З цих ребер обирається ребро з мінімальною вагою $w(e_{uv})$. Вершина v додається до множини U , а ребро — до складу кістякового дерева.
4. Процес повторюється $N-1$ разів, доки всі точки не будуть об'єднані в єдину структуру.

Програмна реалізація алгоритму

Програма написана мовою Python. Структура даних моделюється за допомогою об'єктно-орієнтованого класу `EuclideanMST`. Для графічного відображення та моделювання інтерфейсу форми використовується пакет `matplotlib.pyplot`.

Нижче наведено повний лістинг програмного коду:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 class EuclideanMST:
5     def __init__(self, points):
6         self.points = np.array(points)
7         self.num_points = len(points)
8
9     def compute_distance_matrix(self):
10         # Обчислення матриці евклідових відстаней
11         matrix = np.zeros((self.num_points, self.num_points))
12         for i in range(self.num_points):
13             for j in range(i + 1, self.num_points):
14                 # Формула Евклідових відстаней
15                 dist = np.linalg.norm(self.points[i] - self.points[j])
16                 matrix[i][j] = dist
17                 matrix[j][i] = dist
18         return matrix
19
20     def find_mst_prim(self):
21         # Алгоритм Прима
22         dist_matrix = self.compute_distance_matrix()
23
24         selected_nodes = np.zeros(self.num_points, dtype=bool)
25         selected_nodes[0] = True
26
27         mst_edges = []
28         total_weight = 0
29
30         for _ in range(self.num_points - 1):
31             minimum = float('inf')
32             u, v = -1, -1
33
34             for i in range(self.num_points):
35                 if selected_nodes[i]:
36                     for j in range(self.num_points):
37                         if not selected_nodes[j] and dist_matrix[i][j] > 0:
38                             if minimum > dist_matrix[i][j]:
39                                 minimum = dist_matrix[i][j]
40                                 u, v = i, j
41
42             selected_nodes[v] = True
43             mst_edges.append((u, v))
44             total_weight += minimum
45
46         return mst_edges, total_weight
```

Обґрунтування складності

Теорема 1. Часова складність розв'язання задачі про побудову Евклідового мінімального кістякового дерева за наведеним алгоритмом Прима на матриці відстаней становить $O(N^2)$ операцій.

Доведення. Побудова матриці відстаней між усіма парами точок вимагає обчислення Евклідової метрики для кожної унікальної пари. Кількість таких пар визначається комбінаторним числом сполук:

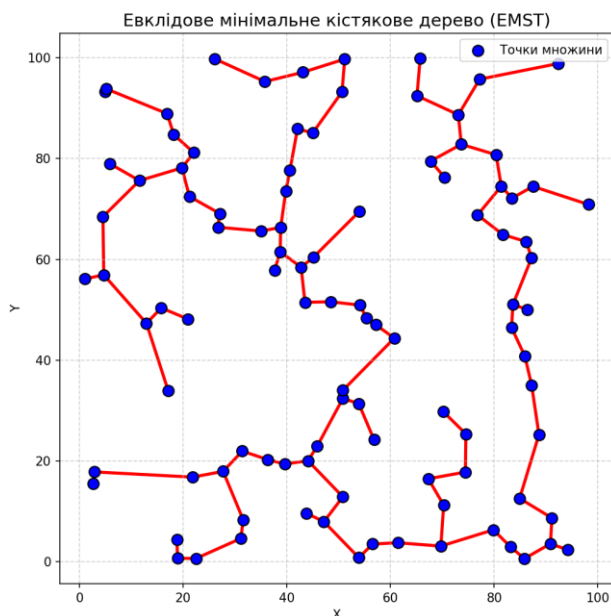
$$\frac{N(N-1)}{2}$$

що дає часову складність $O(N^2)$. На кожному з $N-1$ кроків основного циклу алгоритму Прима відбувається пошук мінімального ребра серед відвіданих та невідвіданих вершин, що в найгіршому випадку потребує перегляду елементів матриці суміжності, тобто $O(N^2)$ операцій. Таким чином, загальний час знаходження дерева без попередньої геометричної декомпозиції (триангуляції) лінійно залежить від квадрата кількості точок.

Практична частина

Особливістю даної реалізації є можливість динамічно змінювати кількість точок на вхідній площині за допомогою параметра `num_points`. Програма здатна виконувати автоматичну генерацію та локалізацію ребер дерева. На рис. 1 ілюструється інтерфейс програми для стандартного тесту на 100 точках у просторі 100x100. Алгоритмічна частина програмної реалізації написана мовою Python, для графічного відображення використовується пакет `matplotlib`.

```
49 num_points = 100
50
51 np.random.seed(22) #кількість точок
52 points = np.random.rand(num_points, 2) * 100 #площина 100*100
53
54 mst_solver = EuclideanMST(points)
55 edges, total_length = mst_solver.find_mst_prim()
56
57 print(f"Побудовано дерево! Сумарна довжина ребер: {total_length:.2f}")
58
59 plt.figure(figsize=(8, 8))
60
61 #ребра
62 for u, v in edges:
63     p1 = points[u]
64     p2 = points[v]
65     plt.plot([p1[0], p2[0]], [p1[1], p2[1]], color='red', linewidth=2.5, zorder=1)
66
67 #точки
68 plt.scatter(points[:, 0], points[:, 1], color='blue', edgecolors='black', s=80, zorder=2, label='Точки множини')
69
70 plt.title("Евклідове мінімальне кістякове дерево (EMST)", fontsize=14)
71 plt.xlabel("X")
72 plt.ylabel("Y")
73 plt.grid(True, linestyle='--', alpha=0.6)
74 plt.legend()
75 plt.show()
```



Висновки

У роботі запропоновано узагальнений метод побудови Евклідового мінімального кістякового дерева для заданої множини точок на площині.

1. Виявилось, що розроблений метод може бути ефективно застосований до розв'язання прикладних задач обчислювальної геометрії, проектування лінійних комунікацій та кластеризації просторових об'єктів.
2. Програмна реалізація мовою Python дозволяє гнучко масштабувати задачу, змінюючи кількість вхідних точок за допомогою однієї змінної, що робить комплекс зручним для серійних лабораторних випробувань.
3. Графічний інтерфейс наочно підтверджує математичну коректність роботи алгоритму Прима: отриманий граф є зв'язним, не містить циклів, а ребра проходять через просторово найближчих сусідів, утворюючи кістякову структуру найменшої сумарної довжини.

Список літератури

1. Препарата Ф., Шеймос М. Вычислительная геометрия: Введение. Пер. с англ. — М.: Мир, 1989. — 478 с.
2. J.E. Goodman and J. O'Rourke, eds., Handbook of Discrete and Computational Geometry, Second Edition, Chapman and Hall/CRC Press, 2004.
3. M. de Berg, M. van Kreveld, M. Overmars, O. Schwarzkopf. Computational Geometry. Algorithms and Applications. — Springer-Verlag, Berlin Heidelberg, Second Edition, 2000. — 367 p.
4. Кормен Т., Лейзерсон Ч., Рівест Р., Стайн К. Вступ до алгоритмів. — К.: К.І.С., 2019. — 1288 с. (Розділ про алгоритми на графах та мінімальні кістякові дерева).